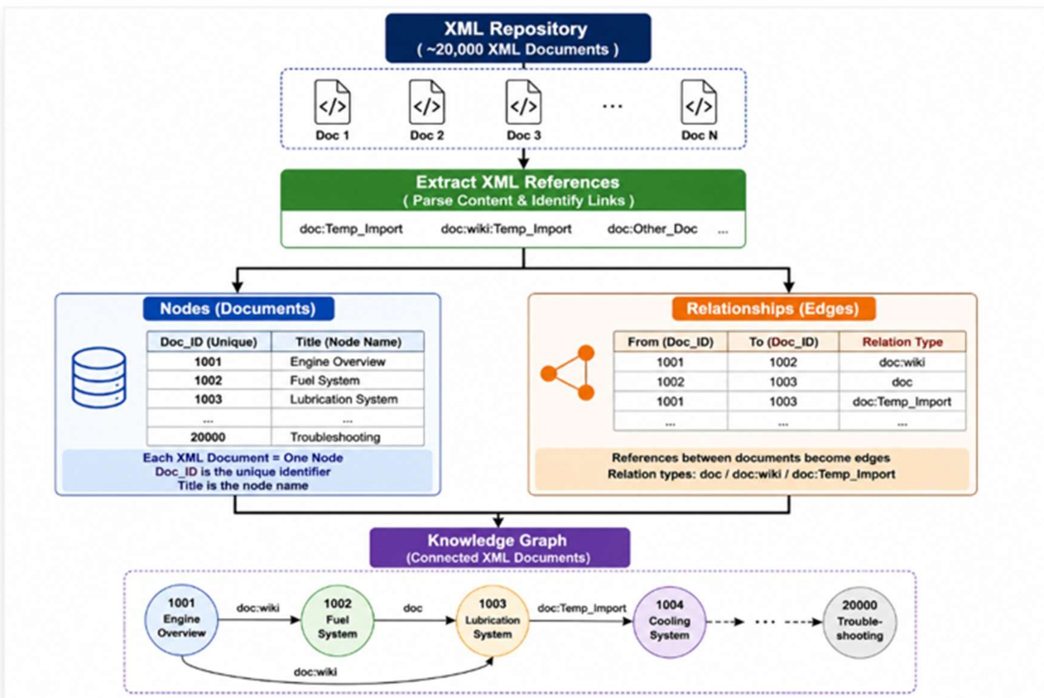
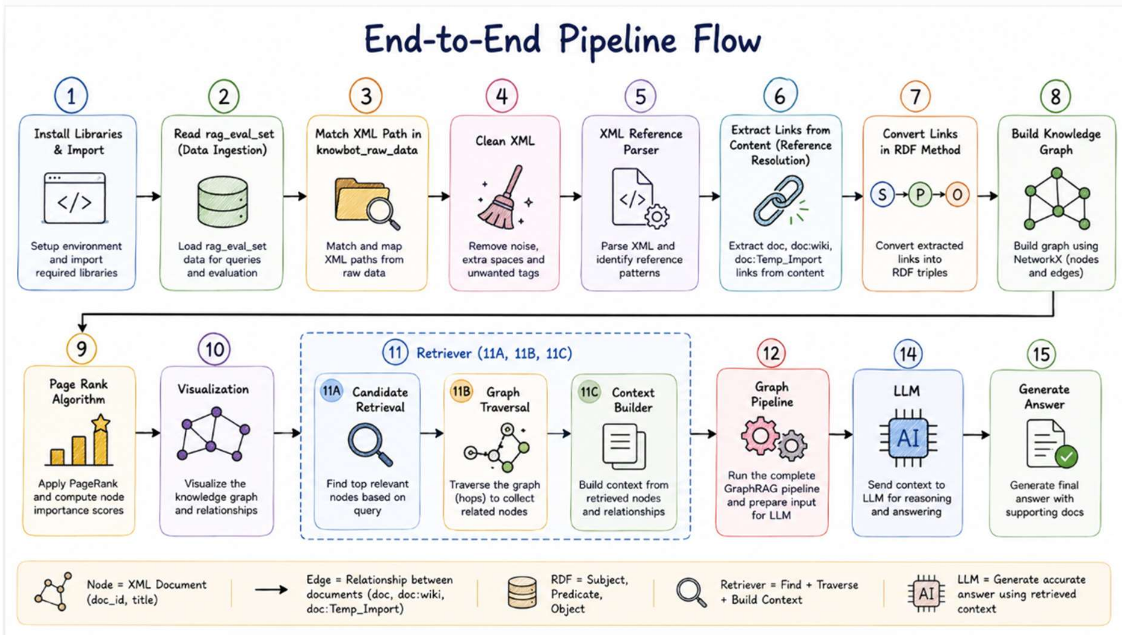
Slide 2:-

Knowledge Graph-based Solution

- Each XML document is converted into a **Knowledge Graph node**.
- **Doc_ID** is used as the unique node identifier.
- The **document title** is used as the node name.
- XML content is scanned to find document references.
- **doc**, **doc:wiki**, and **doc:Temp_Import** links are extracted.
- These links create **relationships (edges)** between documents.
- The final Knowledge Graph connects all related XML documents.



Slide 3:-



Slide 4 :-

Slide 4

Universal XML Reference Parser

Purpose

Extract document references from XML content and identify relationships between XML documents.

Image of output

Key Functions

- Normalize XML references.
- Extract all document links.
- Identify the relationship type.
- Remove duplicate references.
- Store extracted links for RDF generation   keep output

Slide 5 – RDF Triple Generation

Title

RDF Triple Generation

Purpose

Convert the extracted XML references into **RDF triples** that can be used to build the Knowledge Graph.

Slide Content

Steps

- Read all extracted XML references.
- Resolve each reference to its corresponding **Doc_ID**.
- Create an RDF Triple:
  - **Subject** → Parent XML Document
  - **Predicate** → Relationship (doc, doc:wiki, doc:Temp_Import)
  - **Object** → Referenced XML Document
- Ignore missing references and self-loops.
- Store unique RDF triples for Knowledge Graph construction.

Keep image output

Slide 6

Build Knowledge Graph using NetworkX

Purpose

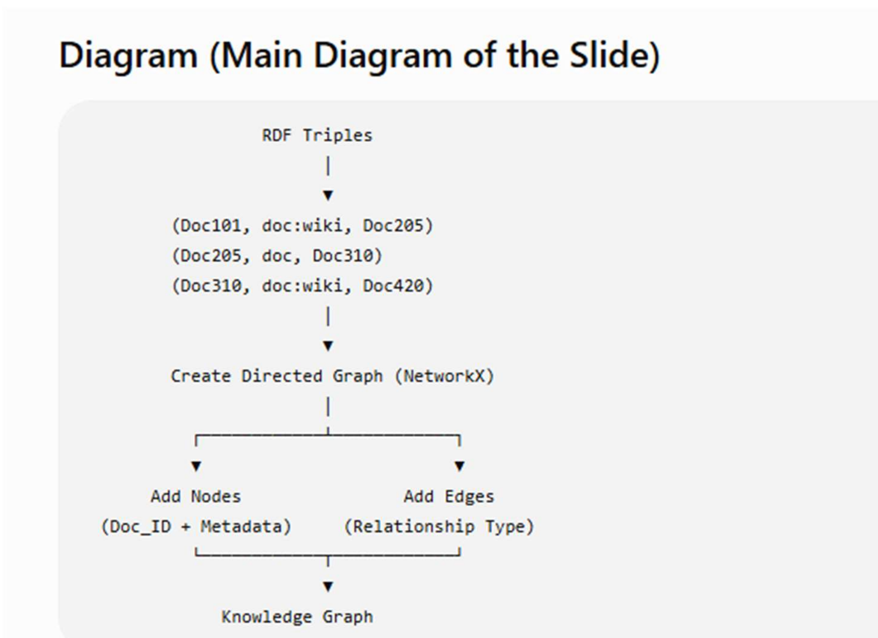
Convert RDF Triples into a **Directed Knowledge Graph** where XML documents are connected through relationships.

Slide Content

Knowledge Graph Construction

- Create a **Directed Graph (DiGraph)** using NetworkX.
- Add each XML document as a **node** using its **Doc_ID**.
- Store document metadata (Title, Reference, Version, Author, Content).
- Convert RDF triples into **graph edges**.
- Validate the graph by checking invalid links and self-loops.
- Generate graph statistics for analysis.

keep visualimage slid 7



slide 8

Slide 8 – GraphRAG Retrieval Pipeline (Cell 11A + 11B)

Title

GraphRAG Retrieval Pipeline

Slide Content

Step 1 – Query Processing

- User enters a technical question.
- Remove stop words and normalize the query.
- Extract important keywords.

↓

Step 2 – Candidate Retrieval (Cell 11A)

- Compare the query with every XML document.
- Calculate the retrieval score using:
  - Keyword Overlap
  - PageRank
  - Degree
  - Title Match
  - Reference Match
- Rank all documents.
- Select the **Top-K candidate documents**.

↓

Step 3 – Graph Traversal (Cell 11B)

- Start traversal from the Top-K documents.
- Perform **Breadth-First Search (BFS)**.
- Explore both:
  - Outgoing relationships
  - Incoming relationships
- Retrieve connected documents within **2 hops**.
- Rank the retrieved nodes.

Slide 9 – Context Builder (Cell 11C)

Title

Knowledge Graph Context Builder

Slide Content

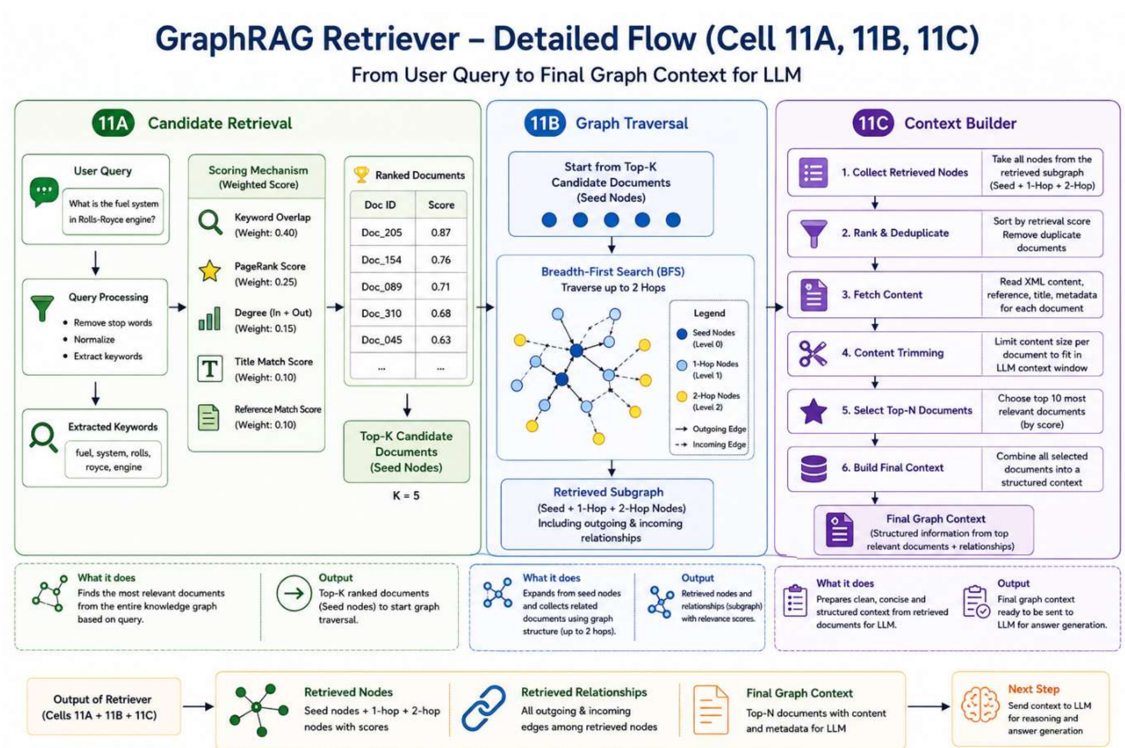
Step 4 – Context Building

- Sort retrieved documents by retrieval score.
- Remove duplicate documents.
- Read XML content from each node.
- Limit the content size.
- Select the Top 10 documents.
- Build one structured context for the LLM.

↓

Output

- Retrieved Documents
- Retrieved Relationships
- Final Graph Context



Slide 11 Title

GraphRAG Retrieval Pipeline

What to write on the slide

Purpose

Combine all retrieval components into a single GraphRAG pipeline.

Pipeline Steps

1. Receive the user query.
2. Retrieve Top-K candidate documents.
3. Traverse the Knowledge Graph.
4. Build the Graph Context.
5. Create the retrieved subgraph.
6. Return the retrieval package to the LLM.

Diagram

```
        User Query
             |
             ▼
    Candidate Retrieval
       (Cell 11A)
             |
             ▼
  Knowledge Graph Traversal
       (Cell 11B)
             |
             ▼
   Graph Context Builder
       (Cell 11C)
             |
             ▼
 Retrieved Subgraph + Context
```

|
▼
GraphRAG Retrieval Result

Output Package

Component	Description
Query	User question
Candidates	Top-K retrieved documents
Retrieved Nodes	Connected documents
Retrieved Subgraph	Graph containing retrieved nodes
Graph Context	Context sent to the LLM

Slide 12

Slide 12 – GraphRAG Retrieval Execution (Cell 14)

Title

GraphRAG Retrieval Execution

Purpose

Execute the complete GraphRAG retrieval pipeline for a user query.

Slide Content

Execution Steps

- User submits a technical query.
- Initialize the GraphRAG pipeline.
- Execute the retrieval process.
- Retrieve relevant documents and relationships.
- Generate the retrieved subgraph.

- Display retrieval statistics.

Diagram

```
        User Query
          |
          ▼
      GraphRAG Pipeline
          |
  ┌───────────┼───────────┐
  ▼       ▼       ▼
Retrieved  Retrieved    Retrieved
Documents  Relationships  Subgraph
  └───────────┼───────────┘
          ▼
     Retrieval Summary
```

Output

- User Query
- Retrieved Documents
- Retrieved Relationships
- Retrieved Subgraph
- Retrieval Statistics

Slide 13 – LLM Prompt & Answer Generation (Cell 15)

Title

LLM Prompt & Answer Generation

Purpose

Generate the final answer using only the retrieved Knowledge Graph context.

Keep
ptompt

output also

Final Architecture

```
                User Query
                    |
        ┌───────────┴───────────┐
        |                       |
        ▼                       ▼
 Semantic Retrieval        Graph Retrieval
  (FAISS Vector DB)      (Knowledge Graph)
        |                       |
        ▼                       ▼
  Top Semantic Docs      Connected Documents
        |                       |
        └───────────┬───────────┘
                    ▼
         Hybrid Retrieval Engine
                    |
     Merge + Remove Duplicates + Rank
                    |
                    ▼
         Hybrid Knowledge Package
                    |
                    ▼
             Azure OpenAI LLM
                    |
                    ▼
      Final Answer + Retrieved Subgraph
```